

Poker Legend

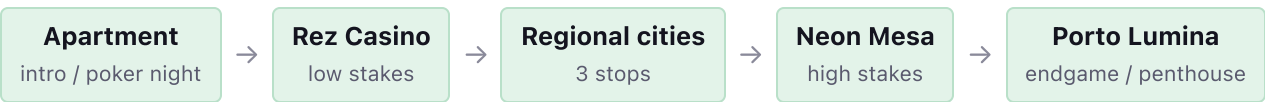
Build roadmap · all phases complete · 29 September 2026

GENRE	PLATFORMS	STACK	VARIANT
Story-driven poker RPG	Browser + iPhone (Capacitor)	Vite · React · TS · PixiJS	Texas Hold'em only
REPO	~/projects/poker-legend		

The concept

LeafGreen-style top-down overworld exploration between story beats, dropping into a fully-simulated, realistic-odds poker engine whenever the player sits down at a table. The player rises from a broke twenty-something at a friend's home game to a poker legend across an escalating ladder of casino cities — risking real debt along the way.

Location ladder



MVP phases

- DONE

1 · Scaffold
Vite + React + TypeScript project, PixiJS wired into a full-viewport canvas, living design docs (GDD + tech design) established as the source of truth.
- DONE

2 · Apartment scene
Walkable room with grid-locked LeafGreen-style movement, tile collision, camera follow, and an NPC (Marcus) whose dialogue triggers the poker night. Later refactored into a reusable overworld component.
- DONE

3 · Poker engine core
Real 52-card deck and shuffling, 5- and 7-card hand evaluation, multi-way side pots, and a full betting-round state machine (blinds, all-ins, street advancement, showdown). AI decisions use Monte Carlo equity estimation — the same technique real equity calculators use — with skill-tiered noise and aggression.
- DONE

4 · Poker night loop
Playable three-handed freezeout against Marcus and Dana. Busting restarts the night from the top; winning all the chips ends the scene and banks the winnings.
- DONE

5 · Bus → local casino
Bus transition carries winnings into a cash wallet. Casino lobby overworld with a Pit Boss (leads to the table) plus ambient slot machines and a craps table. The low-stakes table is a true cash game — fixed buy-in, AI auto-rebuy, human rebuy or leave-and-cash-out.

DONE

6 · Tells & table depth

Opponents give off readable physical cues that correlate with hand strength. Frequency and reliability both scale down with opponent skill, so sharp players give away almost nothing and occasionally give false reads. The UI shows only the observable cue, never its meaning — reading it is the mechanic.

DONE

7 · Economy & persistence

Cash, buy-ins, shops and checkpoint saving. Clothes gate the high rooms and vehicles discount travel, so shops matter beyond flavour.

Beyond the MVP

DONE

8 · Mid-tier cities

Riverbend Landing, Crescent Harbor and Palm Cay — each with its own map, tables, shops, NPCs and side missions, stepping the stakes from \$2/\$5 up to \$10/\$25.

DONE

9 · Sponsors, debt & collectors

NPCs who spot your skill and stake you into bigger private games. Losing staked money creates debt with a deadline; miss it and collectors hunt you — get caught before reaching a new location and you roll back to your last checkpoint.

DONE

10 · Mentor & whales

A mentor NPC teaching real, established poker strategy, and the recurring whale archetype — big money, bad play, obvious tells, wide unpredictable ranges.

DONE

11 · Vegas-parallel

Neon Mesa: a \$25/\$50 main game plus a dress-code-gated \$100/\$200 high roller room, against sharp and elite opposition whose tells are close to invisible.

DONE

12 · Macau-parallel & endgame

Porto Lumina: a \$200/\$400 nosebleed game and the \$250,000 heads-up match against Nadia Okonkwo. Winning takes the penthouse and opens free play.

How this is being built

- **Engine before UI.** The poker engine is plain TypeScript with no React or rendering dependencies, so its correctness can be tested headlessly and trusted before anything is built on top of it.
- **Fuzz testing over spot checks.** Full AI-vs-AI games run across many random seeds, asserting chip conservation after every single action — not just at hand boundaries.
- **Real browser verification.** Each UI phase is driven end-to-end with Playwright before being called done.
- **Living docs.** The design doc and tech design in the repo stay current, so each build request can reference them instead of restating context.

Why the browser testing matters

Phase 4 shipped three real bugs that the unit suite could not see: a stale pot display, a React StrictMode double-invoke that silently ate a blind round's worth of chips on the very first hand, and an effect dependency that could stall the AI's turn entirely. All three only surfaced when the actual app was driven in a real browser.

Still open

- How deep the non-poker casino games go; slots and craps remain ambient flavour.
- Audio direction — not yet specified.
- Character customization depth: appearance only, or stat-like traits?
- Art depth: sprites and sound are generated at runtime and hold together, but a dedicated artist and composer would lift both considerably.
- The iOS app has never been compiled or run. The project scaffolds, syncs, and the production bundle it loads is verified in-browser, but proving the native target needs a machine with full Xcode.